OpenMP Parallelization of a Feedforward Neural Network

Implementation and Experimental Analysis

Parallel Programming Laboratory
Universitat Autònoma de Barcelona

Abstract

This project investigates the shared-memory parallelization of a feedforward neural network using OpenMP. We begin by examining the supplied sequential program, then develop and compare loop-based and task-based parallelization. We evaluate execution time, speedup, efficiency, and scaling, with lightweight checks of application behavior. The report follows development order, linking each decision to code dependencies and measured evidence, while distinguishing observations from hypotheses. Quantitative findings and conclusions will be incorporated after the experiments are completed.

1 Experimental approach

1.1 Objective and initial decision

Our objective is to parallelize the supplied C program using OpenMP and evaluate how its performance changes as we increase the number of threads. We first examine the execution flow and data dependencies, then introduce parallelism in controlled steps. This follows the focus of the course project [1].

We keep the original source recoverable in Git and document any necessary memory-safety or defined-execution corrections separately. The dataset, network configuration, and amount of training work must remain consistent within each performance comparison.

1.2 Planned progression

1. **Sequential reference.** Establish what the supplied program computes and establish a usable performance baseline. Record the execution flow, basic correctness checks, configuration, build, and repeated timings.
2. **Loop-based OpenMP.** Identify loops that can execute concurrently without violating dependencies. Explain data ownership and synchronization, validate against the sequential reference, and measure execution across thread counts.
3. **Task-based OpenMP.** Investigate whether a meaningful task decomposition offers an advantage over loop work sharing. Explain task granularity and dependencies, validate the implementation, and compare it under the same conditions.
4. **Comparison with course references.** Compare scaling with the indicative course results. Match the execution environment where possible and state differences where matching is not possible.

The assignment permits loop-based or task-based OpenMP. We will begin with loop work sharing and introduce tasks after identifying a suitable unit of independent work.

We will also measure each OpenMP implementation with one thread as a supplementary check. Our main speedup and efficiency calculations will use the measured sequential program as their baseline.

1.3 Course reference times and comparison limits

The supplied reference document [2] describes its results as indicative values from a functional, optimized OpenMP implementation. It specifies 10 epochs and reports the execution times in Table 1.

Table 1: Course reference execution times for 10 epochs.

Threads	Wilma `nodo.q`	Wilma `new-nodo.q`	Aolin `aolin.q`
Sequential	40.737795	12.095956	11.235670
2	22.542747	6.873421	6.767024
4	11.730357	4.296542	3.658490
6	8.308845	3.474935	2.502528
12	5.263955	2.555904	2.380782

We do not have the source code for this reference times. We will therefore use these indicative times as context.

Our access to Wilma and Aolin clusters allows measurements on the corresponding nodes. We will first compare sequential timings, then test OpenMP with 1, 2, 4, 6, and 12 threads, stating any differences

from the reference setup.

1.4 Validation and measurement

We will check that the neural network runs correctly and produces results similar to the original program. However, our main focus will be execution time: how much faster the parallel versions run, how efficiently they use the available threads, and how performance changes as we add more threads. We will also check that threads work together correctly so that our timing comparisons are meaningful.

We use the performance metrics defined in the course materials [3]. Let N denote the problem size and p the number of OpenMP threads. Let $T(N, 1)$ be our measured execution time of the sequential program, and let $T(N, p)$ be the execution time of the parallel version being evaluated with p threads. Speedup is:

$$S(N, p) = \frac{T(N, 1)}{T(N, p)}. \tag{1}$$

Efficiency is the achieved speedup divided by the number of threads:

$$E(N, p) = \frac{S(N, p)}{p} = \frac{T(N, 1)}{p\,T(N, p)}. \tag{2}$$

We calculate these metrics from our own sequential and parallel measurements on the same hardware, with matching workloads and timing boundaries. The published course times are used for a separate comparison, not substituted for our measured sequential baseline.

1.4.1 Scaling evaluation

Following the course distinction [3], we will evaluate **strong scaling**: keep the dataset, network configuration, batch size, and number of epochs fixed at a workload N_0, and increase the number of threads through $p \in \{1, 2, 4, 6, 12\}$ where the allocated hardware permits.

We will plot execution time $T(N_0, p)$, speedup $S(N_0, p)$, and efficiency $E(N_0, p)$ against p. The ideal scaling references are:

$$\begin{aligned} T_{\text{ideal}}(N_0, p) &= \frac{T(N_0, 1)}{p}, \\ S_{\text{ideal}}(N_0, p) &= p, \\ E_{\text{ideal}}(N_0, p) &= 1. \end{aligned} \tag{3}$$

Thus, ideally, doubling the thread count halves runtime. We will identify where adding threads gives little further improvement or increases runtime, and report the corresponding speedup and efficiency. Scaling is assessed through these curves, rather than by a separate single value.

Weak scaling instead increases the workload in proportion to the number of threads, keeping work per thread constant; ideally, runtime stays constant. This requires a separate experiment with growing workloads and is not measured by our fixed-workload comparison with the course reference times.

We will report repeated timings and their variability, keeping compiler flags consistent within each comparison. Explanations involving caches, vectorization, or bandwidth will require supporting evidence; otherwise, they remain hypotheses.

2 Understanding the sequential program

2.1 Execution flow and timing boundaries

We begin with `main()` in the supplied `main.c`, which establishes the order of execution [4]:

1. `readConfiguration()` reads the network dimensions, dataset paths, and training parameters. It uses the default configuration file unless another path is supplied as a command-line argument.
2. `init()` allocates the neural network and initializes its weights and biases.
3. `gettimeofday()` records the starting time.
4. `train_neural_net()` loads the training data and trains the network.
5. `test_nn()` loads the test data and evaluates the trained network.
6. `elapsedT()` computes the elapsed time, after which `dinit()` releases the remaining network memory.

This order identifies our first dependency: testing uses the parameters produced by training, so these two phases cannot simply run concurrently. We must look for parallel work within them rather than execute the two function calls in parallel.

The timer starts after configuration and network initialization and stops after testing, before final network cleanup. The reported time therefore includes dataset loading, computation, console output, and input-data cleanup inside the training and testing functions. It is not a training-only measurement. We will assume that this was the timing boundary used for the reference times and we will preserve it for the initial comparisons and label any additional phase timings separately. No OpenMP directives have been introduced and no cluster timings have been collected at this stage.

2.2 Baseline corrections

Before benchmarking, we corrected the following implementation issues:

- **Image loading:** a bounded row buffer prevents

the string terminator from overflowing the pixel array.

- **Random-number generation:** unsigned 32-bit arithmetic replaces signed overflow, with explicit conversion back to the existing signed results, preserving the original's usual 32-bit wraparound sequence.
- **Dataset loading:** training targets are initialized only when loading training data, avoiding unintended writes during test-data loading.
- **Bias handling:** bias gradients are reset and applied to every hidden and output layer, while the input layer is excluded.

We will use these corrections in both the sequential and parallel versions so that their performance comparisons share the same baseline. Updating the output-layer biases can change the network's recognition results.

References

[1] Universitat Autònoma de Barcelona, *Parallel Programming: Practical Laboratory Project*, academic year 2026–2027, especially Sections III and IV. Repository copy.

[2] Universitat Autònoma de Barcelona, *FeedForward Neural Networks: Reference Time – SEQ + OpenMP*, October 2026. Repository copy.

[3] Universitat Autònoma de Barcelona, *Parallel Programming, Lecture 3: Introduction to Parallelism*, slides 20–22 and 26–30, speedup, efficiency, and scalability.

[4] Supplied sequential implementation, `main.c`, functions `main()`, `train_neural_net()`, and `test_nn()`.